WebRTC — Complete Cheat Sheet

Roman Urdu Training Guide · Basic → Advance · 30 Practical Tasks · Full-Stack Developer Track

> **Kaise parhna hai:** Part 0 mein samjho ke call aakhir hoti kaise hai (signaling → ICE → STUN/TURN → P2P) — ye "big picture" hai, isi ke bina code ajeeb lagta hai. Part 1 = WebRTC core API cheat sheet, Part 2 = signaling server (jo Socket.IO se banega — pichhli cheat sheet kaam aayegi!), Part 3 = real patterns. Part 4 = WebSocket/Socket.IO se comparison + STUN vs TURN + mesh vs SFU. Phir **Part 5 ke 30 tasks** banao — T4 (pehli apni call) milestone hai. Part 6 ka 30-day plan, Part 7 interview prep. Rule: **2 browser tabs = tera test lab; har flow DevTools WS tab ke saath dekh.**

Part 0 — Foundation: Ek call hoti kaise hai (ye poora samjho)

Concept	Easy explanation
WebRTC	Browser-to-browser (P2P) **audio / video / data** bhejne ka open standard. Chrome, Firefox, Safari, Edge — sab mein built-in. Media server ke bina direct chalta hai (zyadatar cases mein).
Signaling	Call shuru hone se pehle dono peers ko ek doosre se **metadata exchange** karna hota hai: "mera SDP hai ye", "ye meri ICE candidates hain". WebRTC spec signaling define NAHI karta — **tum khud banao (Socket.IO/WebSocket se — haan, wahi pichhli skill!)**. Media signaling se nahi jata, sirf chhota control data.
SDP	Session Description Protocol — ek text blob jisme likha hota hai: kaunsi codecs hain (VP8/VP9/H264, Opus), media lines, encryption fingerprint. Call ke 2 steps: **Offer (caller)** → **Answer (callee)**.
NAT / problem	Har device apne router ke peeche private IP hai (192.168.x.x). Direct P2P ke liye dono ko apna **public** address pata hona chahiye — aur symmetric NAT wale routers to seedha connection hi nahi hone dete.
STUN	Ek public server — tumhara public IP:port pooch kar batata hai ("tum duniya ko yahan dikhte ho"). Zyada tar simple NAT cases mein STUN hi kaafi hai → P2P ban jata hai.
TURN	**Relay fallback:** jab P2P na ho (strict/symmetric NAT, corporate firewall), media TURN server ke through jaati hai — dono taraf server se connected. **Har production app mein TURN zaroori hai** warna kuch users kabhi connect nahi honge. (P2P fail hone ka insurance.)
ICE	Connection builder: har jagah se candidates gather karo (local IP, STUN se public, TURN se relay), dono taraf **connectivity checks** (STUN binding) chalao, jo pair sabse acha lage woh pakdo. Candidate types: `host` → `srflx` (STUN) → `relay` (TURN).
DTLS-SRTP	Media hamesha encrypted hoti hai (SRTP), keys DTLS se exchange hote hain — WebRTC built-in. TLS/DTLS + SRTP = secure by default.
Transport	Media/data **UDP** par (low latency) — TCP ki retransmission delays real-time ko marti hain. Bhejne wala: SRTP frames; data channel: SCTP over DTLS.

Poora call flow (yaad karo — 8 steps)

```
1. getUserMedia se apni camera/mic stream lo (permission chahiye, HTTPS zaroori)
2. A aur B signaling server (Socket.IO) par room join karte hain
3. A: new RTCPeerConnection → addTrack(stream) → createOffer → setLocalDescription(offer)
4. A: offer signaling se B ko bhejta hai (+ onicecandidate events trickle hote hain)
5. B: setRemoteDescription(offer) → createAnswer → setLocalDescription → answer wapas
6. A: setRemoteDescription(answer)
7. Dono taraf ICE candidates aate rehte hain → addIceCandidate (queue karo jab tak remote description na aaye)
8. ICE checks pass → DTLS handshake → SRTP media chalu — ab data SERVER KE BINA P2P
   (agar P2P fail → TURN relay path use hogi)
```

> **3 zaroori baatein:** (1) Sirf **HTTPS** page par camera + WebRTC chalta hai (secure context). (2) Candidates **remote description se pehle** aa sakte hain — unhe queue karo, warna error. (3) Signaling server sirf *matchmaker* hai — media kabhi usse guzarti nahi (jab tak P2P fail na ho, phir TURN).

Setup (easy way)

```
mkdir webrtc-lab && cd webrtc-lab && npm init -y
npm install socket.io socket.io-client
# local test ke liye self-signed cert banao (ya vite --host + https tunnel jaise ngrok):
# openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 30 -nodes
```

Part 1 — WebRTC Core API Cheat Sheet

1.1 Media lena — getUserMedia / getDisplayMedia

```
// Camera + mic (permission dialog)
const localStream = await navigator.mediaDevices.getUserMedia({
  video: { width: { ideal: 1280 }, height: { ideal: 720 }, facingMode: "user" },
  audio: { echoCancellation: true, noiseSuppression: true },
});
localVideo.srcObject = localStream;

// Mic/Camera toggle — track band/karo (connection waise hi rehta hai)
localStream.getAudioTracks()[0].enabled = false;   // mute
localStream.getVideoTracks()[0].enabled = false;   // cam off

// Screen share
const screen = await navigator.mediaDevices.getDisplayMedia({ video: true });
screen.getVideoTracks()[0].onended = () => { /* user ne share band kiya */ };

// Cleanup — hamesha
localStream.getTracks().forEach(t => t.stop());

// Devices list
const devs = await navigator.mediaDevices.enumerateDevices();   // cameras/mics ke deviceId
```

1.2 Connection — RTCPeerConnection

```
const pc = new RTCPeerConnection({
  iceServers: [
    { urls: "stun:stun.l.google.com:19302" },
    { urls: "turn:turn.example.com:3478", username: u, credential: c },
    // { urls: ["turn:t.example.com:3478?transport=tcp", "turns:t.example.com:5349"] }
  ],
  iceCandidatePoolSize: 10,   // pehle se candidates gather karne ke liye
});

// Tracks bhejne ke liye (caller pehle kare)
localStream.getTracks().forEach(t => pc.addTrack(t, localStream));
// (Transceiver se finer control: pc.addTransceiver("video", { direction: "sendonly" }))

// ---- Events (inhe samjho = WebRTC samjha) ----
pc.onicecandidate = ({ candidate }) => {
  if (candidate) socket.emit("signal", { to: peerId, data: { candidate } });  // trickle
};
pc.ontrack = ({ streams }) => {
  remoteVideo.srcObject = streams[0];       // peer ka media yahan aata hai
};
pc.onconnectionstatechange = () => console.log("pc:", pc.connectionState);
  // new → connecting → connected → disconnected → failed → closed
pc.oniceconnectionstatechange = () => console.log("ice:", pc.iceConnectionState);
  // checking → connected/completed → failed (yahan ICE restart do)
pc.onicecandidateerror = (e) => console.warn(e.errorCode, e.url);   // STUN/TURN fail
pc.onnegotiationneeded = async () => { /* naya track/re-negotiation — T12 */ };
pc.ondatachannel = ({ channel }) => setupChannel(channel);          // receiver side

// Bandwidth control (sender)
const sender = pc.getSenders().find(s => s.track?.kind === "video");
const params = sender.getParameters();
params.encodings = params.encodings || [{}];
params.encodings[0].maxBitrate = 300_000;      // 300 kbps cap
await sender.setParameters(params);              // degradationPreference browser support alag alag hai

// Quality/metrics (2 sec interval se poll karo)
const stats = await pc.getStats();               // reports ka Map
// type: "outbound-rtp" (bytesSent, framesPerSecond) · "inbound-rtp" (packetsLost, jitter)
// "candidate-pair" (currentRoundTripTime, bytes, state) · "remote-inbound-rtp" (loss, rtt)

pc.restartIce();                                 // ICE restart (ya offer {iceRestart:true})
pc.close();                                      // cleanup
```

1.3 Signaling ke saath Offer/Answer (caller + callee dono)

```
// ===== CALLER (A) =====
const pc = new RTCPeerConnection({ iceServers });
localStream.getTracks().forEach(t => pc.addTrack(t, localStream));

const offer = await pc.createOffer();
await pc.setLocalDescription(offer);             // ICE gathering shuru
socket.emit("signal", { to: B, data: { sdp: pc.localDescription } });

pc.onicecandidate = ({ candidate }) => candidate &&
  socket.emit("signal", { to: B, data: { candidate } });

// ===== CALLEE (B) =====
pc.ontrack = ({ streams }) => (remoteVideo.srcObject = streams[0]);

socket.on("signal", async ({ from, data }) => {
  if (data.sdp) {
    await pc.setRemoteDescription(data.sdp);     // offer aaya
    // candidates queue se nikaal kar add karo (jo pehle aaye)
    for (const c of queued) await pc.addIceCandidate(c), queued = [];

    if (data.sdp.type === "offer") {
      const answer = await pc.createAnswer();
      await pc.setLocalDescription(answer);
      socket.emit("signal", { to: from, data: { sdp: pc.localDescription } });
    }
  } else if (data.candidate) {
    if (pc.remoteDescription) await pc.addIceCandidate(data.candidate);
    else queued.push(data.candidate);            // QUEUE — warna error
  }
});
```

1.4 DataChannel — browser-to-browser raw data (bina media ke)

```
// Caller banata hai:
const dc = pc.createDataChannel("chat", { ordered: true });
// Unreliable/fast (games, cursors): { ordered: false, maxRetransmits: 0 }

dc.onopen  = () => dc.send(JSON.stringify({ text: "hi" }));
dc.onmessage = ({ data }) => console.log(data);   // string ya ArrayBuffer
dc.onclose = () => {};
// readyState: "connecting" | "open" | "closing" | "closed"

// Receiver par (bina create kiye):
pc.ondatachannel = ({ channel }) => {
  channel.onmessage = ({ data }) => console.log("peer says:", data);
};
// File transfer: ArrayBuffer chunks — size limit ~256KB/message (practically), slice karke bhejo
```

1.5 API quick table

Kaam	API
Camera/mic	`navigator.mediaDevices.getUserMedia({video, audio})`
Screen	`getDisplayMedia({video:true})`
Connection banana	`new RTCPeerConnection({iceServers})`
Media bhejna	`pc.addTrack(track, stream)` / `addTransceiver(kind, init)`

Negotiation	`createOffer()` → `setLocalDescription()` → signal → `setRemoteDescription()` → `createAnswer()` …
Candidate	`pc.onicecandidate` bhejo · `pc.addIceCandidate(c)` lao (remote desc ke baad)
Peer ka media	`pc.ontrack` → `video.srcObject = streams[0]`
Data bina media ke	`pc.createDataChannel()` / `pc.ondatachannel`
State	`connectionState`, `iceConnectionState`, `signalingState`, `iceGatheringState`
Diagnostics	`pc.getStats()` — RTT, loss, resolution, candidate-pair type
Recovery	`pc.restartIce()` / `createOffer({iceRestart:true})`
Cleanup	`pc.close()` + `stream.getTracks().forEach(t => t.stop())`

Part 2 — Signaling Server Cheat Sheet (Socket.IO se banega)

```
import { Server } from "socket.io";
const io = new Server(httpServer, { cors: { origin: "https://app.example.com" } });

const rooms = new Map();   // room → Set(socketId)

io.on("connection", (socket) => {
  socket.on("join", (room) => {
    socket.join(room);
    socket.data.room = room;
    // Jo pehle se hai usko batao ke naya aaya — wahi "initiator" offer banayega
    const peers = [...(io.sockets.adapter.rooms.get(room) ?? [])].filter(id => id !== socket.id);
    socket.emit("peers", peers);
    socket.to(room).emit("peer-joined", socket.id);
  });

  // ---- SIRF relay karo (SDP + candidates) — media kabhi yahan nahi aati ----
  socket.on("signal", ({ to, data }) => {
    io.to(to).emit("signal", { from: socket.id, data });
  });

  socket.on("disconnect", () => {
    socket.to(socket.data.room).emit("peer-left", socket.id);
  });
});
```

> **Signaling design notes:** (1) Auth: connection par JWT verify karo — warna koi bhi kisi ka call hijack kar sakta hai. (2) Har message mein `from` server khud stamp kare (client ka bheja `from` mat maano). (3) Socket.IO rooms = signaling rooms — perfect match. (4) Signaling **band bhi ho sakta hai** call ke beech mein — media chalta rehta hai; bas renegotiation ke waqt chahiye.

Connection states — kya kahan handle karna hai

State (pc.connectionState)	Kya karo
connecting	UI: "Connecting..." — spinner
connected	Media chal raha hai — video show, call UI live
disconnected	Aksar temporary network blip — 3–5 sec wait, khud theek ho jata hai
failed	ICE fail — 1) `restartIce()` do (ya fresh offer) 2) TURN check karo 3) warna re-call
closed	Sab cleanup — pc.close(), tracks stop, listeners hataye

Production TURN: coturn + temporary credentials

```
# /etc/turnserver.conf
listening-port=3478
fingerprint
use-auth-secret
static-auth-secret=<LONG_RANDOM_SECRET>
realm=example.com
# relay ports: 49152:65535/udp  ·  TLS: listening-port=5349 + certs

// Client ko waqt-bound cred do (signaling server par):
// username = `${expiryTimestamp}:${userId}`
// credential = base64(HMAC-SHA1(secret, username))   → expiry par reset hoti hai
{ urls: "turn:turn.example.com:3478", username, credential }
```

Part 3 — Common Patterns

Pattern 1 — React 1:1 call component ka structure

```
function Room({ roomId, token }) {
  const pcRef = useRef(null);
  const [muted, setMuted] = useState(false);

  useEffect(() => {
    let cancelled = false;
    (async () => {
      const stream = await getUserMedia({ video: true, audio: true });
      localRef.current.srcObject = stream;
      const pc = new RTCPeerConnection({ iceServers });
      pcRef.current = pc;
      stream.getTracks().forEach(t => pc.addTrack(t, stream));
      pc.ontrack = ({ streams }) => { if (!cancelled) remoteRef.current.srcObject = streams[0]; };
      // signaling listeners yahan — offer/answer/candidate (Part 1.3 wala logic)
    })();
    return () => { cancelled = true; pcRef.current?.close();
                   localRef.current.srcObject?.getTracks().forEach(t => t.stop()); };
  }, [roomId]);









  return (/* local + remote
```

> **Safari/iOS:** `<video autoplay playsinline muted>` — local video hamesha `muted` rakho (feedback loop + autoplay policy). Remote video unmute user gesture se. React StrictMode dev mein double-mount = do getUserMedia — cleanup mandatory.

Pattern 2 — Reconnect / ICE restart flow

```
pc.onconnectionstatechange = async () => {
  if (pc.connectionState === "failed") {
    // 1) Pehle soft: ICE restart (naye candidates, wahi peers)
    const offer = await pc.createOffer({ iceRestart: true });
    await pc.setLocalDescription(offer);
    socket.emit("signal", { to: peer, data: { sdp: pc.localDescription } });
  }
  if (pc.connectionState === "disconnected") {
    // 2) 5 sec wait — aksar khud connected ho jata hai
    setTimeout(() => { if (pc.connectionState !== "connected") pc.restartIce(); }, 5000);
  }
};
```

Pattern 3 — Perfect Negotiation (glare se bachna)

```
// Jab dono taraf se saath mein offer aaye (renegotiation = glare).
// Har peer ko "polite/impolite" milta hai (jaise initiator se decide):
//   impolite → naya offer aaye to IGNORE karo
//   polite   → rollback karo aur uska offer accept:  await pc.setLocalDescription({type:"rollback"})
// Socket.IO room ke initiator ko impolite do — bas glare solve.
// (Ful l code: W3C "perfect negotiation" example — T25 mein banoge)
```

Pattern 4 — Mute/raise-hand/state signaling

```
socket.emit("call:state", { mic: false, cam: true });   // peer ko bhejo
socket.on("call:state", ({ mic, cam }) => { peerMutedIcon(!mic); });
// Media band karne ka tareeqa: track.enabled=false (connection active rehta hai)
// Speaker dikhana: remote video par badge — media track nahi, UI state hai
```

Pattern 5 — Mesh vs SFU (group call ka sawaal)

```
2–3 log  : MESH  — har kisi ka sab se direct P2P. Simple, free, kam latency.
              4 clients = 3 uploads + 3 downloads per client = O(n²) streams → 5 log tak hi theek.
4+ log   : SFU   — sab apna stream EK baar SFU server ko bhejte hain, server fans-out karta hai.
              O(n) per client — mediasoup / LiveKit / Janus / mediasoup-server.
              Yahan bhi signaling chahiye + server (relay nahi, selective forward).
Big scale: MCU  — server mix karke ek stream banata hai (recording/webinar) — zyada CPU, zyada delay.
```

Part 4 — Comparisons (ye farq interview mein aate hain)

4.1 WebRTC vs WebSocket / Socket.IO

Point	WebSocket / Socket.IO	WebRTC
Kya bhejta hai	Text/JSON messages — **app data**	Live audio, video + arbitrary data (DataChannel)
Kahan se kahan	Client ↔ **Server** (star topology)	Peer ↔ **Peer** (P2P, server ke through nahi — jab tak TURN na lage)
Transport	TCP (ordered, guaranteed)	UDP + SRTP (low latency, loss tolerate)
Latency focus	Per-message delivery (ms bhi chalega)	Har 20–50ms ka frame — jitter/loss real-time mein handle hote hain
Congestion/Reliability	TCP khud retransmit karta hai (head-of-line blocking)	App decide: data channel reliable (SCTP) ya unreliable (games/cursors)
Setup	Direct connect — simple	Signaling + SDP + ICE + STUN/TURN — negotiation complex
Connection	Server-centric, reconnect = dobara server	Two connections: signaling (WS) + media (P2P) — alag alag
Kis kaam ke liye	Chat messages, notifications, live feeds, **aur WebRTC ka signaling bhi!**	Video/audio calls, screen share, live streaming, P2P file/cursor
Relation	**Dushman nahi, partner hain:** WebRTC call *shuru* karne ke liye signaling WebSocket par hoti hai; media phir P2P jaata hai.	

4.2 STUN vs TURN

	STUN	TURN
Kaam	Sirf tumhara public IP:port batata hai (query)	**Relay** — media apne through forward karta hai
Media server se guzarti hai?	Nahi — P2P ban jata hai	Haan — dono taraf TURN se connection (P2P nahi ho paya)
Cost/bandwidth	MUFT (Google free STUN)	Mehnga — traffic relay = tumhara bandwidth kharch
Kab	Hamesha try pehle (local/normal NAT)	Hamesha config mein rakho as fallback — ~15-20% calls ko iski zaroorat
Failure sign	Symmetric NAT par fail	Bandwidth heavy, but "kaam karta hai hamesha"

4.3 Mesh vs SFU vs MCU

	Mesh	SFU	MCU
Topology	Sab se sab P2P	Sab → server → fans out	Sab → server jo mix karke ek deta hai
Server cost	0 (sirf signaling)	Medium (forward only)	High (decode + mix + encode)
Scalable log	2–4	10–1000+	100s (webinar style)
Latency	Sabse kam	Kam (+ ~100-200ms)	Zyada (mix delay)
Practice	Personal 1:1 / duo	**Production group calls (LiveKit, mediasoup)**	Recording, radio-style

> **Full-stack perspective:** Interview mein kaho — "1:1 = WebRTC mesh with my own signaling; group = SFU (LiveKit/mediasoup) kyunki mesh $O(n^2)$ scale nahi karta; TURN hamesha rakhunga kyunki ~15% networks mein strict NAT hota hai." — yehi senior jawab hai.

Part 5 — Practical Tasks: Basic → Advance (30)

Rules: Pehle bina dekhe likho, phir hint. **2 browser tabs + DevTools (WS tab + console)** = lab. Har kaam HTTPS/local cert ke saath. Har state change console mein log karo — WebRTC "andhere mein type karna" hai, getStats se hi roshni milti hai.

Level 1 — Basic (Week 1) `10 tasks`

T1. Camera/mic permission page: `getUserMedia` se stream lo, local `<video>` mein dikhao, "Stop" button se sab tracks stop karo. Denied permission ka error UI bhi dikhao.

Hint: `srcObject = stream` + `autoplay playsInline muted` · `track.stop()` · try/catch `NotAllowedError`

T2. Mute/Camera toggle: buttons se `track.enabled = false/true` — state icon update. Check karo ke connection/preview chalta rehta hai.

Hint: `stream.getAudioTracks()[0].enabled` — *band* nahi karna, sirf disable.

T3. Signaling server (Socket.IO): `join` (room), `peers` list, `peer-joined` / `peer-left` events, aur `signal` relay (from server stamp kare). 2 tabs se test — events flow DevTools mein dekho.

Hint: Part 2 ka code — abhi sirf logging karo, WebRTC mat lagao. Ye server **poore rest of lab** ka base hai.

T4. 🎯 **Pehli call:** 2 tabs — full offer/answer flow (Part 1.3) → `ontrack` → remote video dikhe. Sirf local video bhejo (peer ka remote bhi aayega — dono taraf same code chalta hai, caller/callee ka farq sirf offer kaun banata hai).

Hint: "peers" list mein pehla = initiator (offer banaye) · dobara join karne wale ko "peer-joined" se pata chale. Debug: `pc.signalingState` console par — "have-local-offer" → "stable".

T5. ICE candidates trickling: `onicecandidate` par har candidate log karo (type dekhoge: host, srflx), receiver par `addIceCandidate`. Remote description na ho to queue dikhao.

Hint: candidate.candidate string mein "typ srflx"/"typ relay" likha hota hai · queue array pattern Part 1.3 mein.

T6. States observer: `onconnectionstatechange`, `oniceconnectionstatechange`, `onicecandidateerror` sab log karo. Phir test: ek tab ka network band karo (DevTools offline) → disconnected → failed dekho.

Hint: state table Part 2 mein — UI badge ("Connecting/Connected/Reconnecting") isi se banao.

T7. STUN verify: `pc.getStats()` se candidate-pair nikalo aur dekho ke connection **host/srflx** (P2P) par hai ya relay. STUN server change karke (Google → your own) candidates list compare karo.

Hint: `stats.forEach(r => r.type === "candidate-pair" && r.state === "succeeded")` → local/remote candidate ids se type pata karo.

T8. DataChannel chat: `createDataChannel("chat")` — text messages peer-to-peer bhejo/jao, UI mein bubbles. Connection ke bina typing band ho, message input check.

Hint: caller `createDataChannel`, callee `ondatachannel` · data channel bina media ke bhi chalta hai (mic off call = sirf data channel test).

T9. Screen share: `getDisplayMedia` → `pc.replaceTrack(oldTrack, newTrack, stream)` se live video replace (renegotiation ke bina!). User share band kare to original camera wapas.

Hint: `sender.replaceTrack(screenTrack)` · `onended` par wapas `replaceTrack(cameraTrack)`.

T10. Cleanup drill: Call band button → `pc.close()`, tracks stop, socket leave, UI reset — aur dobara call join karke sab wapas chalao (leak nahi — camera light band honi chahiye).

Hint: useEffect cleanup ya explicit function — "dobara join" hi asli test hai.

Level 2 — Intermediate (Week 2–3) 10 tasks

T11. Full React 1:1 video call app: room URL (`/room/:id`), join/leave buttons, remote video, mute/cam/screen controls, state badge — Part 3 Pattern 1 structure.

Hint: pcRef/streamRef · cleanup mandatory (StrictMode double-mount) · signaling events useEffect mein subscribe, return mein off.

T12. Renegotiation: call ke beech mein "Add participant" ya toggle se naya track add karo → `onnegotiationneeded` fire hoga — offer/answer dobara chalao (ya perfect negotiation lagao).

Hint: `pc.addTrack` ke baad event aata hai — simple flow: naya offer bhejo (bina collision ke). Collision T25 mein.

T13. Failure handling: `onicecandidateerror` log karo (STUN 701 errors dikhenge), phir `connectionState === "failed"` par automatic **ICE restart** chalao (max 2 tries) — call wapas aani chahiye.

Hint: Part 3 Pattern 2 · signaling bhi chahiye (naya offer) — restart sirf ek taraf se nahi hota.

T14. TURN setup: `coturn` install (Docker: `coturn/coturn`) → config (Part 2) → client iceServers mein add → T7 wali getStats se **relay** candidate verify karo (P2P blocked karne ke liye firewall/rules test ya `iceTransportPolicy: "relay"` force option).

Hint: `new RTCPeerConnection({ iceTransportPolicy: "relay" })` = sirf TURN use karo — testing ke liye best trick. Bandwidth TURN par badhta hua dekho getStats mein.

T15. Call states in UI: "Connecting / Connected / Reconnecting / Call ended" — peer ka mic-off/cam-off badge (Pattern 4 signaling) + join/leave toasts.

Hint: pc state events + socket events dono se state machine banao (ek reducer kaafi).

T16. getStats dashboard: 2 sec interval par dikhao — RTT, packets lost, FPS, resolution, candidate pair type (host/srflx/relay), bytes sent/received. Bandwidth kam karke numbers badalta hua dekho.

Hint: outbound-rtp → framesPerSecond, frameWidth/Height · candidate-pair → currentRoundTripTime (seconds), bytesSent · remote-inbound-rtp → packetsLost.

T17. Bandwidth control: `sender.setParameters` se `maxBitrate` (100kbps ↔ 2Mbps) toggle — getStats mein bitrate change verify. Network slow profile (DevTools throttle) par UI suggestion: "Video quality lowered".

Hint: params.encodings[0].maxBitrate = 100_000 · bit rate ≈ (bytes ka delta × 8) / seconds

T18. DataChannel file transfer: file ko ArrayBuffer chunks (64KB) mein todo — progress bar, receive side reassemble + download link. 5MB file test.

Hint: `file.arrayBuffer()` → slices → `dc.send(chunk)` · metadata (name, size, total chunks) pehle bhejo · end par `Blob` banao.

T19. Unreliable channel: do channels banao — "reliable" (ordered) aur "cursor" (`ordered:false, maxRetransmits:0`). Network throttle par farq observe: unreliable drop ho jata hai, reliable wait karta hai.

Hint: options createDataChannel ke 2nd argument mein · use case: live cursor/typing (drop OK) vs chat (guaranteed).

T20. Recording: `MediaRecorder(localStream)` se call record karo (pause/resume/stop) → WebM file download. Stop par camera light band + pc active — dono independent hain.

Hint: `new MediaRecorder(stream, {mimeType:"video/webm"})` · ondataavailable chunks · onstop → Blob → URL.createObjectURL.

Level 3 — Advance (Week 4) 10 tasks

T21. Mesh group call (3–4 peers): har peer ke liye alag RTCPeerConnection (Map: peerId → pc) — join/leave par add/remove. 3 tabs se test, sab ek doosre ko dekhein.

Hint: joiner offer banata hai already-joined peers ko (iska initiator rule) · remote streams map rakho · **scale note:** 5 log ke baad mesh toot'ta hai — isliye T22.

T22. SFU ka concept demo: LiveKit/mediasoup ka quickstart chalao — publish (1 uplink) + subscribe model samjho, mesh se farq (bandwidth per client) numbers se compare karo.

Hint: LiveKit — `npx livekit-cli` + SDK test room · sawaal: 4 log mesh mein kitne streams vs SFU mein kitne? (16 vs 4)

T23. Codec preferences: `RTCRtpTransceiver.setCodecPreferences` se H264 vs VP8 choose karo, SDP mein codec line verify karo. Kab kaunsa? (H264 = device/HW support, VP8/9 = web).

Hint: `pc.getTransceivers()` + RTCRtpReceiver.getCapabilities("video").codecs filter → preferences mein daal ke renegotiate.

T24. Simulcast basics (SFU context): `sender.setParameters` mein multiple `encodings` (rid: 0/1/2 — low/med/high resolution) — SFU viewer bandwidth ke hisaab se layer chunta hai (lyr='simulcast').

Hint: params.encodings = [{scaleResolutionDownBy:4, rid:"q"},{rid:"h"},{scaleResolutionDownBy:2, rid:"f"}] — har rid alag res (browser/SFU support check karo).

T25. Perfect negotiation implement karo: polite/impolice rule (initiator = impolite), glare par rollback — dobara-dobara renegotiation karo (add/remove track) bina crash ke.

Hint: Part 3 Pattern 3 + W3C perfect negotiation example · `makingOffer` flag, `setLocalDescription()` bina args ke (auto offer/answer), ignoreOffer flag.

T26. TURN temporary credentials: coturn `use-auth-secret` + signaling server par HMAC-SHA1 se waqt-bound cred generate (expiry 5 min) — client ko har naye connection par dedo, expire hone par refresh.

Hint: username = `${Math.floor(Date.now()/1000 + 300)}:${userId}` · credential = base64(hmac(sha1, secret, username)) · Node: `crypto`.

T27. Monitoring: har call ke end par getStats snapshot (duration, avg bitrate, packet loss %, relay use %) signaling server par bhejo — ek simple "call quality report" log/dashboard.

Hint: start/end par stats lo, delta nikaalo · kisi bhi call mein loss > 5% ya relay > 50% = flag.

T28. Security audit: (1) signaling auth JWT (2) server from-stamp (3) room membership check — "koi bahar wala signal bheje to drop" (4) TURN creds time-bound (5) recording consent (UI checkbox). Har point fix code ke saath.

Hint: `socket.data.userId` se verify karo ke receiver isi room ka member hai — warna 4001-style reject.

T29. Mobile/Safari test: iOS Safari par `playsinline` + user-gesture start, `enumerateDevices` se camera switch (facingMode/environment), background/foreground (page visibility) par track handling. 1 real phone + Chrome Remote/device mode.

Hint: Safari mein autoplay unmuted video blocked — local video muted rakho, remote un mute button se · `deviceId` getUserMedia constraint mein do.

T30. Capstone: Production-grade 1:1 calling feature — JWT signaling + rooms + ICE (STUN + TURN relay tested) + mute/cam/screen + DataChannel chat + reconnect (ICE restart) + getStats quality badge + cleanup + Part 7 checklist pass. *Ye ho jaye to WebRTC interview + production dono ready hain.*

Hint: T4 + T11 + T13 + T14 + T16 ka integration — pehle event/flow diagram banao (kaunsa event kya karega), phir code.

Part 6 — 30-Day Learning Plan

Days	Kya seekhna hai	Kya banao
Day 1–3	Foundation: signaling, SDP, ICE, STUN/TURN, NAT — Part 0	Diagram khud banao (8-step flow) · 2-3 articles + is PDF repeat
Day 4–5	getUserMedia, tracks, toggle, getDisplayMedia	T1–T2: camera page + controls
Day 6–8	Socket.IO signaling server (pehchana!)	T3: join/peers/signal relay — DevTools WS frames
Day 9–11	Offer/answer, ICE candidates, ontrack	🎯 T4–T5: **pehli P2P call** (milestone!) + trickle ICE
Day 12–14	States, errors, cleanup, DataChannel, screen share	T6–T10: observer + data chat + replaceTrack
Day 15–17	React integration, renegotiation, UI states	T11–T12: React call app (week ka main project)
Day 18–21	TURN (coturn), ICE restart, getStats, bandwidth	T13–T16: relay verify + auto-recovery + stats dashboard
Day 22–24	DataChannel advanced, recording, bitrate control	T17–T20: file transfer + MediaRecorder
Day 25–27	Mesh, SFU concept, codecs, simulcast	T21–T24: 3-peer mesh + LiveKit quickstart
Day 28–29	Perfect negotiation, TURN creds, security, mobile	T25–T29: polish + audit
Day 30	Capstone + interview prep	T30 + Part 7 ke 10 sawaal khud jawab do

Practice rhythm (roz ka nizam)

- **Roz 1–1.5 ghanta:** 30 min concept + 60 min task. **2 tabs + console + WS tab** hamesha khula rakho.
- Har flow **2 baar** likho: ek baar simplified (sirf P2P), ek baar production (TURN + restart + states).
- **getStats se dosti** karo — WebRTC mein bina stats ke guess karna = bug dhoondna andhere mein.
- Har naye feature ke baad 1 baar `iceTransportPolicy:"relay"` test — TURN path bina real firewall ke test hota hai.

Part 7 — Production Checklist + Trainer Tips + Interview

12-point production checklist (deploy se pehle):

1. **HTTPS/WSS** mandatory — getUserMedia secure context ke bina chalta hi nahi.
2. **STUN + TURN dono** — sirf STUN = kuch calls kabhi connect nahi hongi; coturn (ya managed: Twilio/Cloudflare Calls) hamesha.
3. **Signaling auth** (JWT) + server-side `from` stamp + room membership check — call hijack se bachna.
4. **State-driven UI:** connecting/connected/disconnected/failed ke alag screens; "reconnecting" dikhao, andher mein mat chhodo.
5. **ICE restart + retry limits** (max 2–3) — infinite retry se battery + logs kharab.
6. **Cleanup hamesha:** `pc.close()` + `track.stop()` — warna camera light ON reh jayegi (user trust!).
7. **getStats monitoring:** RTT, packet loss, relay % — quality alerts + debugging ke liye.
8. **Bandwidth guard:** maxBitrate cap + degrade policy — low-end devices/fair networks par courtesy.
9. **Group calls:** 4+ log = SFU (LiveKit/mediasoup) — mesh mat becho production mein.
10. **TURN credentials time-bound** (HMAC + expiry) — permanent creds leak hone par relay fraud.
11. **Safari/iOS quirks:** playsinline, muted local, user gesture — QA matrix mein iPhone zaroor.
12. **Recording consent** + data policy — recording ka pata participants ko ho (legal).

Interview ke 10 sawaal (pehle khud jawab do)

Sawaal	Ek line answer
WebRTC kya hai, kaise kaam karta hai?	Browsers ka P2P audio/video/data standard — signaling se SDP/ICE exchange, phir STUN/TURN se connectivity, media SRTP (encrypted) par UDP se directly
Signaling kyun chahiye jab P2P hai?	Call shuru hone se pehle capability/addresses exchange (SDP offer-answer + ICE candidates) — WebRTC spec signaling define nahi karta, WebSocket se khud banao
STUN vs TURN?	STUN = public address poochta hai (P2P possible); TURN = relay fallback jab NAT strict ho — media server se guzarti hai, hamesha kaam karta hai but mehnga
ICE kya karta hai?	Saare candidates (host/srflx/relay) gather karke dono taraf connectivity checks chalata hai, best pair choose karta hai
SDP offer/answer?	Caller offer (meri media capabilities) bhejta hai, callee answer — dono agree karte hain codecs/session par; phir ICE
Connection failed to kya karoge?	Check getStats (candidate type) → ICE restart (max 2-3) → TURN path verify → UI "reconnect" state; na ho to fresh negotiation
Mesh vs SFU?	Mesh = sab P2P (2-4 log, $O(n^2)$ bandwidth); SFU = sab ek server ko, server fans-out (O(n), 4+ log — LiveKit/mediasoup)
WebRTC secure kaise hai?	Media DTLS-SRTP se encrypted (built-in), page HTTPS compulsory; signaling auth app ko karna hai
WebSocket vs WebRTC?	WebSocket = client-server text messages (TCP); WebRTC = peer-peer media/data (UDP, low latency) — signaling ke liye WebSocket hi use hota hai
DataChannel kya hai?	WebRTC ka "private TCP/UDP pipe" browser-to-browser — reliable ya unreliable (maxRetransmits) — files, games, cursors ke liye, server se bina guzre

---